MINI PROJECT | FINAL EVALUATION

Real-Time Network Intrusion Detection System

using Packet Inspection and Web-Based Alerting

Rohan Pandey

Roll No. 2501720100117 | Section 2D

Guide: Mr. Vikas, Assistant Professor

Department of Computer Science & Engineering & Allied Branches

Lloyd Institute of Management and Technology | Session 2026-27

Problem Statement

Small networks have no real-time way to see attacks as they happen.

Where: homes, hostels and college labs

Today: attacks pass unseen in the packets

Gap: tools are manual, complex or unexplainable

Need: light, explainable, alerts within a second

ATTACKS DETECTED

Port Scan

tries every port to find an open one

SYN Flood

floods a server with half-open connections

ARP Spoofing

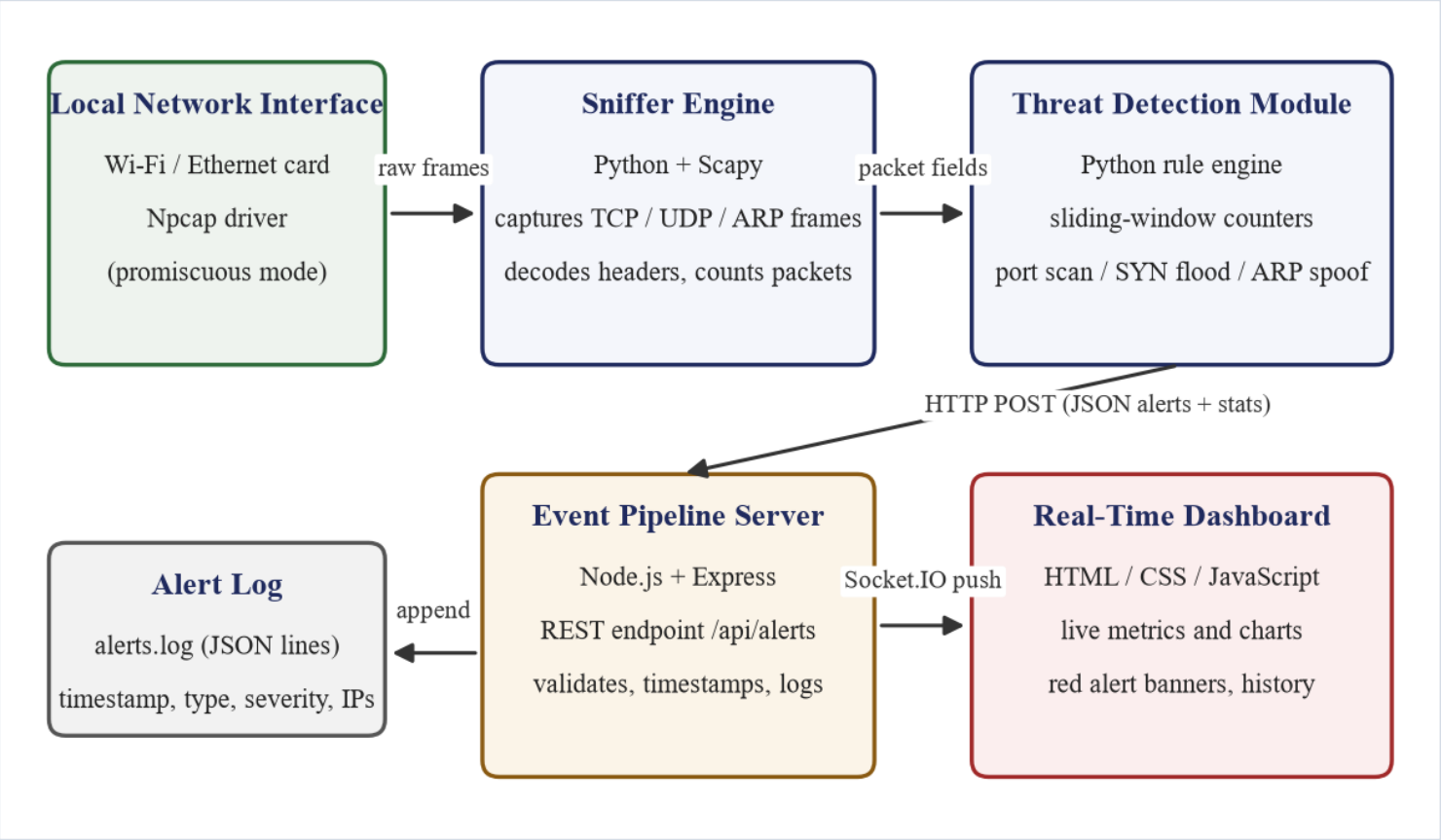
puts the attacker in the middle of traffic

Objectives: All Achieved

- ✓ Capture live TCP, UDP and ARP packets
- ✓ Detect port scans with a sliding-window rule
- ✓ Detect SYN floods from half-open handshakes
- ✓ Detect ARP spoofing from a trust table
- ✓ Alert pipeline to a Node.js and Express server
- ✓ Live web dashboard, updated without refresh
- ✓ Automated tests and measured performance

All seven objectives were met. All three attack rules are implemented, tested and shown live on the dashboard.

Architecture and Technology



Packets flow left to right; alerts flow from the detector to the browser. **Python captures, Node.js serves, the browser displays.**

TECHNOLOGY STACK

Npcap 1.89
raw packets on Windows

Python 3.13 + Scapy 2.8
decodes IP, TCP, UDP, ARP

Python standard library
rules: deque window, set of ports

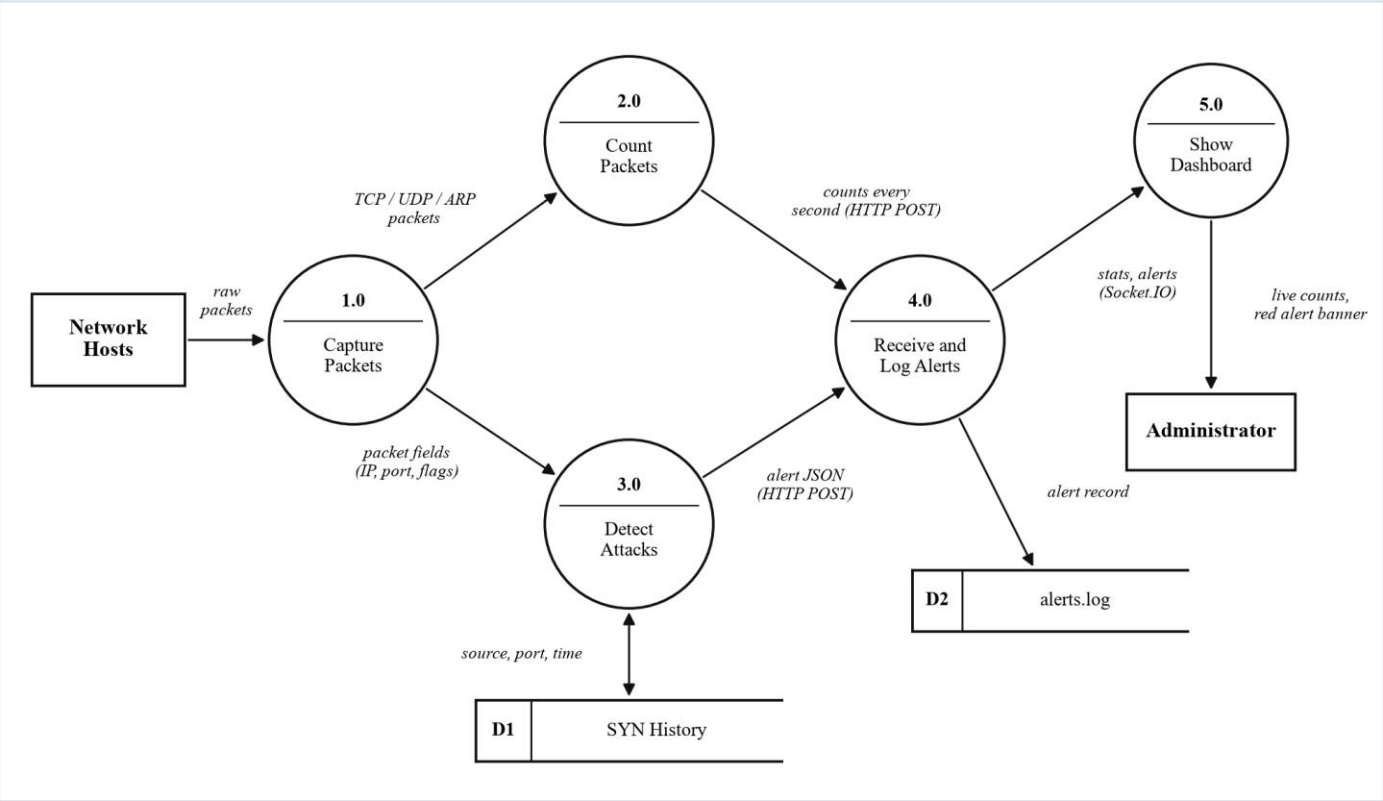
Node.js 24 + Express 5
receives alerts by HTTP POST

Socket.IO 4
pushes alerts to the browser

HTML, CSS, JS + Chart.js
live chart, table and banner

unittest, Nmap, Wireshark
testing and verification

System Design



Level-1 Data Flow Diagram

DESIGN ARTEFACTS

- Context and level-1 DFDs
- Use-case diagram
- Sequence diagram
- Component diagram
- Deployment diagram
- Flowchart of the port-scan rule

All diagrams are in the report, **numbered chapter-wise** as Fig. 3.1 to 4.2.

Detection Rules

Port Scan

> 15 distinct ports

from one source

within 5 seconds

medium

SYN Flood

100+ half-open SYNs

to one host

few completed

high

ARP Spoofing

gateway IP seen

with a different

MAC address

critical

All rules are deterministic. Each keeps a few seconds of history in a sliding window, compares a count against a threshold, and raises an alert with a cool-down so one attack gives one alert.

Testing

UNIT TESTS

10 / 10 tests passing
one per rule, with
positive and negative cases

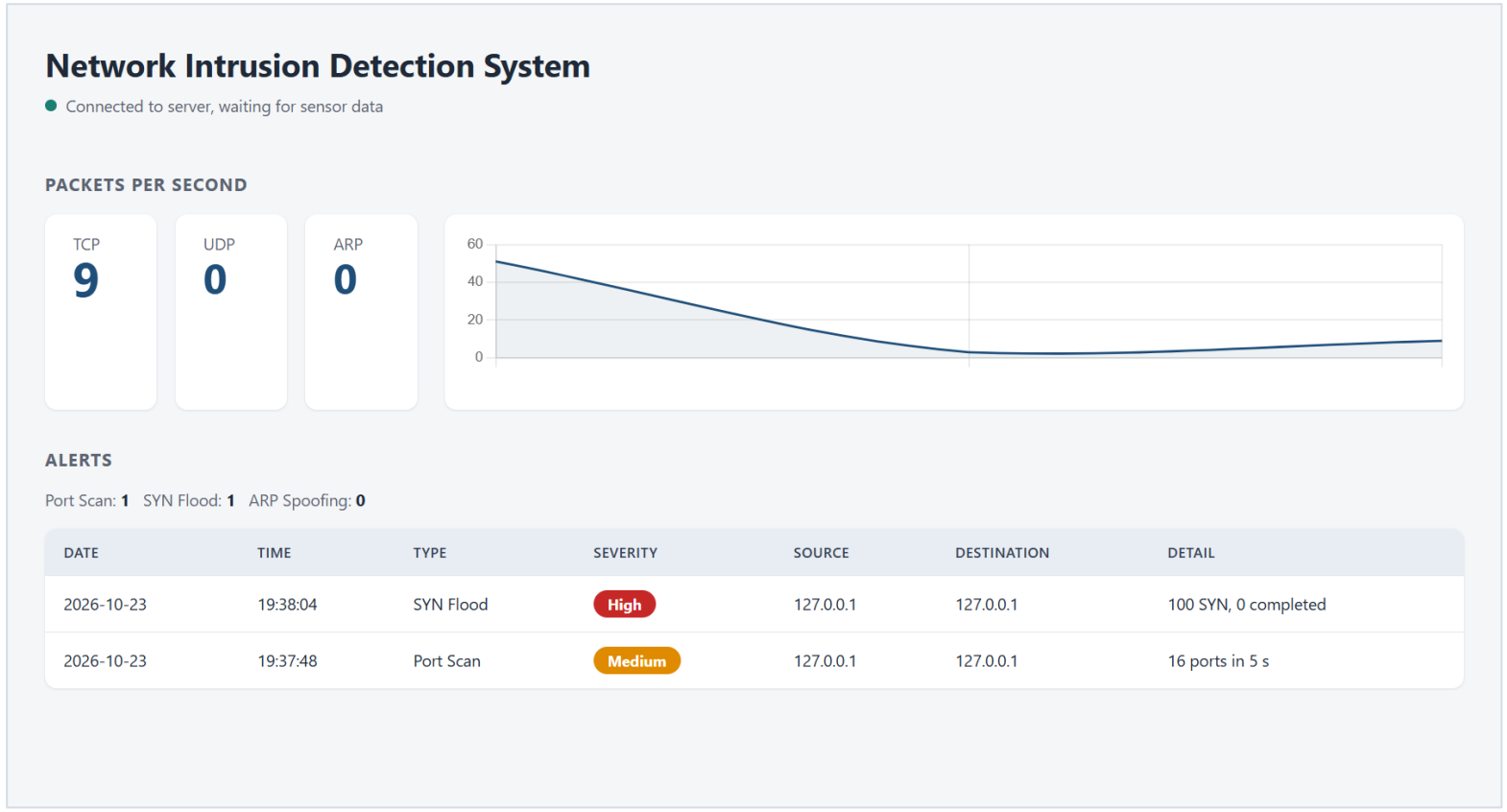
INTEGRATION TESTS

Real port scan and SYN flood against the host
Alert reached the dashboard and log in a few ms
No false alarm during normal browsing

What the negative tests confirm

- Reply (SYN-ACK) packets are not a scan
- Completed handshakes are not a flood
- Genuine ARP replies raise no alarm
- Slow activity below the threshold stays silent
- A port scan is not mistaken for a flood
- A cool-down stops duplicate alerts

Results: Live Dashboard



Dashboard after detecting a port scan and a SYN flood

3 attacks detected

~2 ms detection latency

10/10 unit tests pass

~200 lines of code

Conclusion and Future Scope

A complete, working NIDS. All objectives met: live capture, three transparent detection rules, an alert pipeline, and a real-time dashboard, verified by ten unit tests and live attacks, in about 200 lines of code.

KEY STRENGTHS

- ✓ Every alert traceable to a threshold
- ✓ Real-time: under one second
- ✓ Small, readable, easy to extend
- ✓ Fails safely if the server is down

FUTURE SCOPE

- More rules: ICMP flood, DNS tunnelling
- E-mail or messaging notifications
- Store alerts in SQLite, searchable
- Authenticate the sensor to the server
- Active response: firewall the attacker

Thank You

Questions are welcome

github.com/rohanpandey436/realtime-nids

Rohan Pandey | Roll No. 2501720100117 | Guide: Mr. Vikas